Homework 11

The `chardev_crypto_module.c` code is a Linux kernel module that creates a character device (`/dev/char_dev`)..

Key Components

The provided code is a Linux kernel module named "chardev_crypto_module" that creates an input/output character device with added functionality for encryption using the SHA-256 hashing algorithm. The module defines various functions, including device_open, device_release, device_read, device_write, device_ioctl, and additional functions related to SHA-256 encryption.

The encryption functionality is implemented in the `encrypt` function, which takes a plaintext string, computes its SHA-256 hash, and stores the result in the `message` buffer. The SHA-256 hash is then displayed using the `show_hash_result` function.

The `device_write` function handles the writing of data to the device. It copies the user-provided data from the buffer to a kernel buffer, and then the `encrypt` function is called to perform encryption and update the `message` buffer with the encrypted result.

The module registers the character device during initialization using the `register_chrdev` function and creates a corresponding device file in the /dev directory. This file is found under `/dev/char_dev`.

During module cleanup, the device and class are destroyed, and the character device is unregistered.

Please note that the code assumes the availability of the SHA-256 hashing algorithm in the kernel and uses cryptographic functions from the kernel's crypto API. The encryption process involves computing the SHA-256 hash of the plaintext and storing the result in the `message` buffer. The resulting hash is then displayed in hexadecimal format.

Linux Machine Terminal Commands

On the linux machine running the kernel module we use the following commands to compile and insert the kernel module:

```
make && sudo insmod chardev_crypto_driver.ko
```

Next we write a string to the character device and after it has been processed, we can read the hash back from the same device.

```
echo “Hello from the user” > /dev/char_dev

cat /dev/char_dev
```

Lastly we use the OpenSSL s_client to send this hash to the Raspberry Pi over a secure SSL connection.

```
cat char_dev | openssl s_client -connect 192.168.254.152:44330
```

Raspberry Pi OpenSSL Server

We have an OpenSSL server running on the Raspberry PI. This server listens for encrypted messages sent from our second linux machine. First we created certificate files to enable a secure connection:

```
openssl req -new -newkey rsa:2048 -days 365 -nodes -x509
-keyout server.key -out server.crt
```

Then we started a server which listens for connections:

```
openssl s_server -key server.key -cert server.crt -accept
44330
```

In the following console output we can see the message sent from the other machine which is the hash previously generated by the kernel module. The red highlighted text is the hashed string.

```
-----BEGIN SSL SESSION PARAMETERS-----

MH0CAQECAgMEBAITAgQg0dpa35emJjmHnUf68LocgZCntN1ZzHKZz8+NN+48U/UE

ME1tY/J0T/A1gGZ+4RwvbKGBmzbNqbITIU6fKelfVTkKeRMqCjnAbGs5oHnIX+FC

86EGAgRlb+cOogQCAhwgpAYEBAEAAACuBgIEaIk97w==

-----END SSL SESSION PARAMETERS-----

Shared
ciphers:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128_GCM
_SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:DHE-RSA-A
ES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:
DHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GC
M-SHA256:DHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-SHA384:ECDHE-RSA-AES2
56-SHA384:DHE-RSA-AES256-SHA256:ECDHE-ECDSA-AES128-SHA256:ECDHE-RSA-AES128-
SHA256:DHE-RSA-AES128-SHA256:ECDHE-ECDSA-AES256-SHA:ECDHE-RSA-AES256-SHA:DH
```

E-RSA-AES256-SHA:ECDHE-ECDSA-AES128-SHA:ECDHE-RSA-AES128-SHA:DHE-RSA-AES128
-SHA:AES256-GCM-SHA384:AES128-GCM-SHA256:AES256-SHA256:AES128-SHA256:AES256
-SHA:AES128-SHA

Signature Algorithms:
ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed448:RSA-PSS+SHA256:RSA-PSS
+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA+SHA
256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224:DSA+SHA224:DSA+SHA256:DSA
+SHA384:DSA+SHA512

Shared Signature Algorithms:
ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed448:RSA-PSS+SHA256:RSA-PSS
+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA+SHA
256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224

Supported Elliptic Groups:
X25519:P-256:X448:P-521:P-384:0x0100:0x0101:0x0102:0x0103:0x0104

Shared Elliptic groups: X25519:P-256:X448:P-521:P-384

CIPHER is TLS_AES_256_GCM_SHA384

Secure Renegotiation IS supported

6fd37ea244821a0c4eb47e7c1c363bb7dbd1235257f559a36b82ad28f12b431d

shutting down SSL

CONNECTION CLOSED

-----BEGIN SSL SESSION PARAMETERS-----

MH0CAQECAgMEBAITAgQgWfuG/KDocMtpp/HncVs7r+zl/a+2UFCaGX8vq9ckNrcE

MC8D2pOt6dY/mXzHRn9aIsSfLdilOe6M475xDNFzCLPC3MUSCpZkcGGV4x1bTmnY

+qEGAgRlb+cbogQCAhwgpAYEBAEAAACuBgIEVNF1xw==

-----END SSL SESSION PARAMETERS-----

Shared
ciphers:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128_GCM
_SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:DHE-RSA-A
ES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:
DHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GC
M-SHA256:DHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-SHA384:ECDHE-RSA-AES2
56-SHA384:DHE-RSA-AES256-SHA256:ECDHE-ECDSA-AES128-SHA256:ECDHE-RSA-AES128-
SHA256:DHE-RSA-AES128-SHA256:ECDHE-ECDSA-AES256-SHA:ECDHE-RSA-AES256-SHA:DH
E-RSA-AES256-SHA:ECDHE-ECDSA-AES128-SHA:ECDHE-RSA-AES128-SHA:DHE-RSA-AES128
-SHA:AES256-GCM-SHA384:AES128-GCM-SHA256:AES256-SHA256:AES128-SHA256:AES256
-SHA:AES128-SHA

```
Signature Algorithms:
ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed448:RSA-PSS+SHA256:RSA-PSS
+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA+SHA
256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224:DSA+SHA224:DSA+SHA256:DSA
+SHA384:DSA+SHA512

Shared Signature Algorithms:
ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed448:RSA-PSS+SHA256:RSA-PSS
+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA+SHA
256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224

Supported Elliptic Groups:
X25519:P-256:X448:P-521:P-384:0x0100:0x0101:0x0102:0x0103:0x0104

Shared Elliptic groups: X25519:P-256:X448:P-521:P-384

CIPHER is TLS_AES_256_GCM_SHA384

Secure Renegotiation IS supported

6fd37ea244821a0c4eb47e7c1c363bb7dbd1235257f559a36b82ad28f12b431d

shutting down SSL

CONNECTION CLOSED
```